BEM-VINDO AO PROJETO

Visão geral

CONHECIMENTO EM PRÁTICA

Clínica Vida+

Gestão que cuida. *Aprendizado que transforma.*

Um projeto acadêmico que conecta tecnologia, organização e cuidado em cinco passos.



05 etapas integradas	02 regras de atendimento	16 combinações lógicas	03 relações «include»

O QUE VOCÊ VAI ENCONTRAR

Uma jornada em cinco passos.

Explore cada etapa e acompanhe como as partes se conectam.

01

GESTÃO ÁGIL

Planejamento de sprints

Organize tarefas e acompanhe a evolução em um quadro visual.

02

PROGRAMAÇÃO

Cadastro em Python

Um programa executável no terminal para cadastrar e consultar pacientes.

03

LÓGICA

Regras de atendimento

Visualize as expressões e todas as combinações das tabelas-verdade.

04

ESTRUTURA DE DADOS

Fila de pacientes

Entenda a ordem de chegada com pseudocódigo e uma demonstração FIFO.

05

MODELAGEM

Casos de uso

Visualize atores, responsabilidades e relações obrigatórias no sistema.

INTRODUÇÃO

Um projeto, diferentes perspectivas.

A Clínica Vida+ é um cenário fictício para integrar práticas de gestão de projetos e fundamentos de desenvolvimento de software. Cada etapa resolve uma parte do problema: organizar o trabalho, estruturar um cadastro e representar decisões e fluxos.

O quadro é uma demonstração local, e o programa Python funciona no terminal. Nenhuma parte desta aplicação deve receber dados pessoais ou médicos reais.

OBJETIVOS

O que vamos construir.

- Planejar e acompanhar atividades em sprints.
- Aplicar listas e dicionários em um cadastro Python.
- Validar regras por meio da lógica proposicional.
- Demonstrar a estrutura FIFO e modelar casos de uso.

CONCLUSÃO

Conhecimento conectado, do início ao fim.

O trabalho une planejamento, implementação e análise formal: a consulta normal é liberada em 3 combinações e a emergência em 6; a fila preserva a ordem de chegada e o diagrama explicita responsabilidades e funcionalidades obrigatórias.

REFERÊNCIAS E FERRAMENTAS

Para ir além.

Construído com HTML5, CSS3, JavaScript e Python 3; quadro armazenado no navegador e versionamento com Git.

Documentação Python (https://docs.python.org/pt-br/3/tutorial/datastructures.html)

MDN Web Docs (https://developer.mozilla.org/pt-BR/docs/Learn_web_development)

Guia do Trello (https://support.atlassian.com/trello/docs/creating-a-new-board/)

Scrum Guide (https://scrumguides.org/scrum-guide.html)

Especificação UML (https://www.omg.org/spec/UML/2.5.1/)

Documentação Git (https://git-scm.com/doc)

PASSO 01 / GESTÃO ÁGIL

Planejamento de sprints.

Da ideia à entrega: visualize tarefas, acompanhe o progresso e registre aprendizados.

Este quadro é uma demonstração local **inspirada no Trello**; não é um quadro criado na plataforma. Para usar o Trello real, **veja como criar um quadro**.

SPRINT EM FOCO

Sprint 01 — Planejamento

Compreender o cenário da clínica e transformar os requisitos em um plano de trabalho.

02/03/2026 a 13/03/2026 | 12 dias corridos (inclusive)

Em andamento

PROGRESSO DA SPRINT **40%**

2 de 5 concluídas 6/10 itens do checklist

Quadro de tarefas 5 tarefas

Arraste os cartões ou use o seletor para mover uma tarefa.

Relatórios de todas as sprints

RELATÓRIO DE SPRINT — Sprint 01 — Planejamento
Clínica Vida+ · Projeto acadêmico · Dados fictícios

Objetivo: Compreender o cenário da clínica e transformar os requisitos em um plano de trabalho.
Status da sprint: Em andamento
Período: 02/03/2026 a 13/03/2026
Duração: 12 dia(s) corrido(s), incluindo as duas datas
Progresso: 2 de 5 tarefa(s) concluída(s) (40%).
Checklist: 6 de 10 item(ns) concluído(s).

RESULTADOS
Tarefas concluídas: 2. Tarefas ainda não concluídas: 3.
Requisitos iniciais organizados e quadro de trabalho preparado para a equipe fictícia.

DESAFIOS
Detalhar os critérios de aceite sem perder de vista o escopo da atividade.

PONTOS POSITIVOS
A divisão das tarefas facilitou a visualização das prioridades.

MELHORIAS
Revisar riscos e estimativas com antecedência na próxima sprint.

TAREFAS E ACOMPANHAMENTO

1. Planejar testes de aceitação
   Status: Backlog
   Prazo: 13/03/2026
   Descrição: Definir cenários para validar as entregas de cada etapa.
   Checklist:
   • [ ] Identificar cenários de borda
   • [ ] Revisar critérios de aceite
   Comentários:
   • 05/03/2026, 12:00 — Listar casos de erro e lista vazia.

2. Detalhar critérios de aceite
   Status: Sprint Atual
   Prazo: 11/03/2026
   Descrição: Descrever como verificar as funcionalidades obrigatórias.
   Checklist:
   • [x] Registrar requisitos funcionais
   • [ ] Revisar critérios com a equipe
   Comentários:
   • Nenhum comentário.

3. Modelar fluxos de consulta
   Status: Em Progresso
   Prazo: 12/03/2026
   Descrição: Definir atores, interações e dependências do sistema.
   Checklist:
   • [x] Identificar atores
   • [ ] Desenhar casos de uso
   Comentários:
   • 09/03/2026, 14:30 — Representar as inclusões obrigatórias.

4. Levantar os requisitos
   Status: Concluído
   Prazo: 04/03/2026
   Descrição: Separar os cinco passos e suas entregas.
   Checklist:
   • [x] Ler a atividade
   • [x] Listar entregáveis
   Comentários:
   • 04/03/2026, 13:00 — Escopo inicial consolidado.

5. Montar o quadro da sprint
   Status: Concluído
   Prazo: 06/03/2026
   Descrição: Organizar as atividades nas quatro listas de acompanhamento.
   Checklist:
   • [x] Criar listas de status
   • [x] Distribuir tarefas
   Comentários:
   • Nenhum comentário.

RELATÓRIO DE SPRINT — Sprint 02 — Desenvolvimento
Clínica Vida+ · Projeto acadêmico · Dados fictícios

Objetivo: Construir as soluções e validar os cálculos e as entregas do projeto.
Status da sprint: Em andamento
Período: 16/03/2026 a 27/03/2026
Duração: 12 dia(s) corrido(s), incluindo as duas datas
Progresso: 1 de 4 tarefa(s) concluída(s) (25%).
Checklist: 4 de 8 item(ns) concluído(s).

RESULTADOS
Tarefas concluídas: 1. Tarefas ainda não concluídas: 3.
Protótipo de cadastro elaborado e regras de atendimento organizadas para verificação.

DESAFIOS
Conferir todas as combinações e os casos de entrada inválida.

PONTOS POSITIVOS
As verificações foram divididas em pequenas tarefas independentes.

MELHORIAS
Automatizar mais cenários antes da apresentação final.

TAREFAS E ACOMPANHAMENTO

1. Revisar a apresentação final
   Status: Backlog
   Prazo: 27/03/2026
   Descrição: Conferir documentação e impressão de todos os passos.
   Checklist:
   • [ ] Revisar referências
   • [ ] Conferir versão para impressão
   Comentários:
   • Nenhum comentário.

2. Validar tabelas-verdade
   Status: Sprint Atual
   Prazo: 23/03/2026
   Descrição: Conferir as 16 combinações para cada modalidade.
   Checklist:
   • [x] Gerar combinações
   • [ ] Revisar contagem
   Comentários:
   • 20/03/2026, 12:00 — Verificar o caso sem agendamento e sem pagamento.

3. Construir diagrama de casos de uso
   Status: Em Progresso
   Prazo: 25/03/2026
   Descrição: Associar atores e relacionamentos include.
   Checklist:
   • [x] Desenhar limite do sistema
   • [ ] Revisar setas include
   Comentários:
   • Nenhum comentário.

4. Criar cadastro de pacientes
   Status: Concluído
   Prazo: 20/03/2026
   Descrição: Implementar menu e operações no terminal usando Python.
   Checklist:
   • [x] Criar menu em loop
   • [x] Tratar entradas inválidas
   Comentários:
   • 20/03/2026, 15:20 — Dados mantidos somente em memória.

PASSO 02 / PROGRAMAÇÃO

Cadastro em Python.

Um programa de terminal de verdade, organizado com listas e dicionários.

> Os registros ficam **somente na memória** durante a execução e são apagados ao sair. Use exclusivamente dados fictícios; o programa não é adequado a dados reais.

O PROGRAMA

O essencial, com clareza.

O arquivo `clinica_vida.py` usa uma lista de dicionários para guardar nome, idade e telefone de cada paciente. Seu menu permanece em execução até a opção de sair.

01 **Cadastro validado**
Valida nome, idade inteira de 0 a 130 e telefone com DDD.

02 **Estatísticas confiáveis**
Total, idade média e pessoas mais novas e mais velhas, inclusive empates.

03 **Busca e listagem**
Busca parcial sem diferenciar maiúsculas ou acentos e trata resultados vazios.

Terminal · exemplo de execução — PYTHON 3

```
$ python3 clinica_vida.py

Use apenas dados fictícios. Os dados são apagados ao

================================================
          CLÍNICA VIDA+ | CADASTRO
================================================
1. Cadastrar paciente
2. Ver estatísticas
3. Buscar paciente pelo nome
4. Listar todos os pacientes
5. Sair
------------------------------------------------
Escolha uma opção:
```

Exemplo ilustrativo · execute o arquivo no seu terminal

COMO EXECUTAR

Pronto para começar?

Baixe o arquivo, abra o terminal na pasta onde ele foi salvo e execute um dos comandos abaixo.

macOS / LINUX	`python3 clinica_vida.py`
WINDOWS	`py clinica_vida.py`

Código-fonte completo

O código exibido é o mesmo do arquivo Python disponibilizado.

```
#!/usr/bin/env python3
"""Cadastro acadêmico de pacientes da Clínica Vida+.

Os dados ficam apenas na memória e são descartados ao sair. Utilize somente
informações fictícias: este programa não oferece persistência nem segurança
para dados pessoais ou médicos reais.
```

```
"""

import re
import unicodedata


MENU = """
================================================
          CLÍNICA VIDA+ | CADASTRO
================================================
1. Cadastrar paciente
2. Ver estatísticas
3. Buscar paciente pelo nome
4. Listar todos os pacientes
5. Sair
------------------------------------------------"""


def normalizar(texto):
    """Ignora caixa e acentos nas buscas por nome."""
    decomposicao = unicodedata.normalize("NFKD", texto.strip().casefold())
    return "".join(caractere for caractere in decomposicao
                   if not unicodedata.combining(caractere))


def solicitar_nome():
    while True:
        nome = input("Nome: ").strip()
        if nome and any(caractere.isalpha() for caractere in nome):
            return nome
        print("Nome inválido. Informe um nome que contenha letras.")


def solicitar_idade():
    while True:
        texto = input("Idade (0 a 130): ").strip()
        try:
            idade = int(texto)
        except ValueError:
            print("Idade inválida. Digite um número inteiro entre 0 e 130.")
            continue
        if 0 <= idade <= 130:
            return idade
        print("Idade inválida. Digite um número inteiro entre 0 e 130.")


def solicitar_telefone():
    while True:
        telefone = input("Telefone com DDD (10 ou 11 dígitos): ").strip()
        digitos = re.sub(r"\D", "", telefone)
        if re.fullmatch(r"[0-9()+\-\s]+", telefone) and len(digitos) in (10, 11):
            return telefone
        print("Telefone inválido. Informe 10 ou 11 dígitos, incluindo o DDD.")


def cadastrar_paciente(pacientes):
    print("\n--- CADASTRAR PACIENTE ---")
```

```
    nome = solicitar_nome()
    idade = solicitar_idade()
    telefone = solicitar_telefone()
    pacientes.append({"nome": nome, "idade": idade, "telefone": telefone})
    print(f"Paciente '{nome}' cadastrado(a) com sucesso.")


def calcular_estatisticas(pacientes):
    """Retorna os indicadores ou None quando não há pacientes."""
    if not pacientes:
        return None
    menor_idade = min(paciente["idade"] for paciente in pacientes)
    maior_idade = max(paciente["idade"] for paciente in pacientes)
    return {
        "total": len(pacientes),
        "media": sum(paciente["idade"] for paciente in pacientes) / len(pacientes),
        "mais_novos": [p for p in pacientes if p["idade"] == menor_idade],
        "mais_velhos": [p for p in pacientes if p["idade"] == maior_idade],
    }


def nomes_e_idades(pacientes):
    return "; ".join(f"{p['nome']} ({p['idade']} anos)" for p in pacientes)


def mostrar_estatisticas(pacientes):
    print("\n--- ESTATÍSTICAS ---")
    estatisticas = calcular_estatisticas(pacientes)
    if estatisticas is None:
        print("Nenhum paciente cadastrado. Cadastre um paciente para ver as estatísticas.")
        return
    media = f"{estatisticas['media']:.2f}".replace(".", ",")
    print(f"Total de pacientes: {estatisticas['total']}")
    print(f"Idade média: {media} anos")
    print(f"Mais novo(s): {nomes_e_idades(estatisticas['mais_novos'])}")
    print(f"Mais velho(s): {nomes_e_idades(estatisticas['mais_velhos'])}")


def buscar_pacientes(pacientes, termo):
    """Busca parcial sem diferenciar maiúsculas ou acentos."""
    chave = normalizar(termo)
    if not chave:
        return []
    return [p for p in pacientes if chave in normalizar(p["nome"])]


def exibir_pacientes(pacientes):
    for indice, paciente in enumerate(pacientes, start=1):
        print(f"{indice}. Nome: {paciente['nome']} | Idade: {paciente['idade']} "
              f"| Telefone: {paciente['telefone']}")


def buscar_por_nome(pacientes):
    print("\n--- BUSCAR PACIENTE ---")
    if not pacientes:
        print("Nenhum paciente cadastrado para buscar.")
        return
```

```
    termo = input("Nome ou parte do nome: ").strip()
    if not termo:
        print("Busca inválida. Informe um nome.")
        return
    encontrados = buscar_pacientes(pacientes, termo)
    if not encontrados:
        print("Nenhum paciente encontrado para esse nome.")
        return
    print(f"Resultado(s) encontrado(s): {len(encontrados)}")
    exibir_pacientes(encontrados)


def listar_pacientes(pacientes):
    print("\n--- TODOS OS PACIENTES ---")
    if not pacientes:
        print("Nenhum paciente cadastrado.")
        return
    exibir_pacientes(pacientes)


def main():
    pacientes = []  # Lista de dicionários; os dados existem apenas nesta execução.
    print("Use apenas dados fictícios. Os dados são apagados ao sair.")
    while True:
        print(MENU)
        opcao = input("Escolha uma opção: ").strip()
        if opcao == "1":
            cadastrar_paciente(pacientes)
        elif opcao == "2":
            mostrar_estatisticas(pacientes)
        elif opcao == "3":
            buscar_por_nome(pacientes)
        elif opcao == "4":
            listar_pacientes(pacientes)
        elif opcao == "5":
            print("Programa encerrado. Até mais!")
            break
        else:
            print("Opção inválida. Escolha um número de 1 a 5.")


if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nEntrada encerrada. Programa finalizado sem salvar dados.")
```

PASSO 03 / LÓGICA PROPOSICIONAL

Regras de atendimento.

Quatro condições, duas modalidades e todas as 16 combinações possíveis.

A	Tem agendamento
B	Documentos válidos
C	Médico disponível
D	Pagamentos em dia

MODALIDADE 01

Consulta normal

$(A \wedge B \wedge C) \vee (B \wedge C \wedge D)$

3 de 16 situações liberam o atendimento.

MODALIDADE 02

Emergência

$C \wedge (B \vee D)$

6 de 16 situações liberam o atendimento.

EXPERIMENTE

V = verdadeiro · F = falso

Analise uma combinação.

Alterne as condições para ver o resultado e localizar a linha nas tabelas.

A	B	C	D
Agendamento	Documentos	Médico	Pagamentos
F	V	V	F

CONSULTA NORMAL — **Não liberada**

(A ∧ B ∧ C) = F · (B ∧ C ∧ D) = F

EMERGÊNCIA — **Liberada**

C = V · (B ∨ D) = V

? **O caso solicitado: A=F, B=V, C=V, D=F**

Consulta normal: F. O primeiro termo é falso porque não há agendamento (A=F); o segundo é falso porque os pagamentos não estão em dia (D=F). **Emergência: V.** O médico está disponível (C=V) e os documentos são válidos (B=V), portanto B ∨ D é verdadeiro.

TODAS AS POSSIBILIDADES

Tabelas-verdade completas.

A linha destacada corresponde à combinação selecionada acima.

01 / CONSULTA NORMAL

(A ∧ B ∧ C) ∨ (B ∧ C ∧ D)

3 liberadas

A	B	C	D	A∧B∧C	B∧C∧D	Resultado
V	V	V	V	V	V	**V**
V	V	V	F	V	F	**V**
V	V	F	V	F	F	**F**
V	V	F	F	F	F	**F**
V	F	V	V	F	F	**F**
V	F	V	F	F	F	**F**
V	F	F	V	F	F	**F**
V	F	F	F	F	F	**F**
F	V	V	V	F	V	**V**
F	**V**	**V**	**F**	**F**	**F**	**F**
F	V	F	V	F	F	**F**
F	V	F	F	F	F	**F**
F	F	V	V	F	F	**F**
F	F	V	F	F	F	**F**
F	F	F	V	F	F	**F**
F	F	F	F	F	F	**F**

16 combinações · **3 situações liberadas**

02 / EMERGÊNCIA

C ∧ (B ∨ D)

6 liberadas

A	B	C	D	B∨D	Resultado
V	V	V	V	V	**V**
V	V	V	F	V	**V**
V	V	F	V	V	**F**
V	V	F	F	V	**F**
V	F	V	V	V	**V**
V	F	V	F	F	**F**
V	F	F	V	V	**F**
V	F	F	F	F	**F**
F	V	V	V	V	**V**
F	**V**	**V**	**F**	**V**	**V**
F	V	F	V	V	**F**
F	V	F	F	V	**F**
F	F	V	V	V	**V**
F	F	V	F	F	**F**
F	F	F	V	V	**F**
F	F	F	F	F	**F**

16 combinações · **6 situações liberadas**

PASSO 04 / ESTRUTURA DE DADOS

Fila de pacientes.

Uma demonstração da estrutura FIFO: o primeiro a entrar é o primeiro a sair.

01 PRIMEIRO A ENTRAR → 01 PRIMEIRO A SAIR

Novos pacientes entram no fim da fila. O atendimento retira sempre quem está no início, mantendo a ordem dos demais.

ALGORITMO

FIFO

Pseudocódigo

```
INÍCIO
    fila ← lista vazia

    PARA i DE 1 ATÉ 3 FAÇA
        LEIA nome
        LEIA cpf
        ENFILEIRE(fila, { nome: nome, cpf:
    FIM PARA

    paciente_atendido ← DESENFILEIRE(fila)
    ESCREVA "Paciente em atendimento: ", p
    ESCREVA "Pacientes restantes, em ordem
    PARA CADA paciente EM fila FAÇA
        ESCREVA paciente.nome, paciente.cp
    FIM PARA
FIM
```

São inseridos três pacientes, cada um com nome e CPF. Em seguida, o primeiro é removido e os dois restantes são mostrados na ordem original.

DEMONSTRAÇÃO

3 na fila

A fila em ação

Início da fila → Fim da fila

01 **Paciente A** — CPF 000.000.000-00 — PRÓXIMO

02 **Paciente B** — CPF 111.111.111-11

03 **Paciente C** — CPF 222.222.222-22

Paciente A será atendido primeiro.

Nomes ilustrativos e CPFs propositalmente inválidos.

Exemplo esperado: entram Paciente A, Paciente B e Paciente C. Após atender A, permanecem **Paciente B → Paciente C**, nessa ordem.

PASSO 05 / MODELAGEM DE SISTEMAS

Casos de uso.

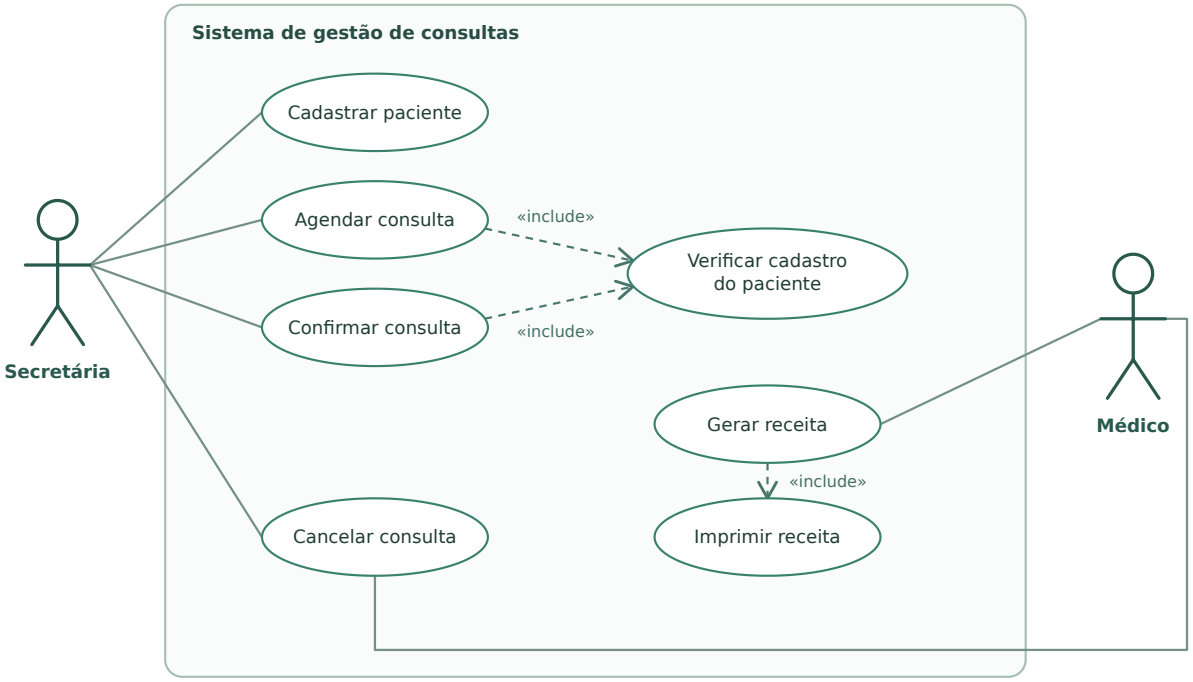
Uma visão clara de quem interage com o sistema e das ações obrigatórias.

DIAGRAMA UML

UML

Sistema de gestão de consultas

Os atores estão fora dos limites do sistema; os casos de uso estão dentro.



Linhas contínuas representam associações. As setas tracejadas apontam do caso de uso principal para o caso incluído.

ATORES

Responsabilidades

S

Secretária

Cadastrar paciente; agendar, confirmar e cancelar consulta.

M

Médico

Cancelar consulta e gerar receita.

RELACIONAMENTOS

Inclusões obrigatórias

«include» indica que o caso incluído é executado obrigatoriamente pelo caso principal.

Agendar consulta «include» → Verificar cadastro do paciente

Confirmar consulta «include» → Verificar cadastro do paciente

Gerar receita «include» → Imprimir receita